



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 2

Principles of Object Modelling

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 2 of 16



Contents

1. Week 2 Overview	3
2. Learning Outcomes	3
3. What Is Object Modelling?	4
4. Why Do We Need Object Models?	4
5. The Principles of Object Modelling	5
6. Object Identification	7
7. Attributes, Behaviour & Responsibility	9
8. Managing Complexity	10
9. Modelling Viewpoints	11
10. Standards for Objects & Modelling	12
11. Worked Examples	12
12. Object-Model Quality & Checklists	14
13. Practical & Tutorial Activities	15
14. Independent Learning (10 Hours)	15
15. Week 2 Summary	16
16. Examination-Style Questions	16
17. References and Further Reading	17

1. Week 2 Overview

Week 1 established the foundation: **business problem** → **business need** → **requirements** → **understanding the problem domain** → **candidate objects**. This week moves one step further.

The official course outline identifies six Week 2 topics: **Principles of Object Modelling**, **Definition of an Object Model**, **Identification of Objects in a System**, **Standards for Objects**, **Managing Complexity** and **Modelling**.

The purpose is to teach students how to think about a system as a collection of meaningful objects and relationships, before the complete UML language is introduced in Weeks 3 and 4.

A model is deliberately simpler than the real system. The Object Management Group (OMG) explains that modelling helps people work at a higher level of abstraction by hiding unnecessary detail and focusing attention on particular aspects of a system — an idea that is absolutely central to OOAD.

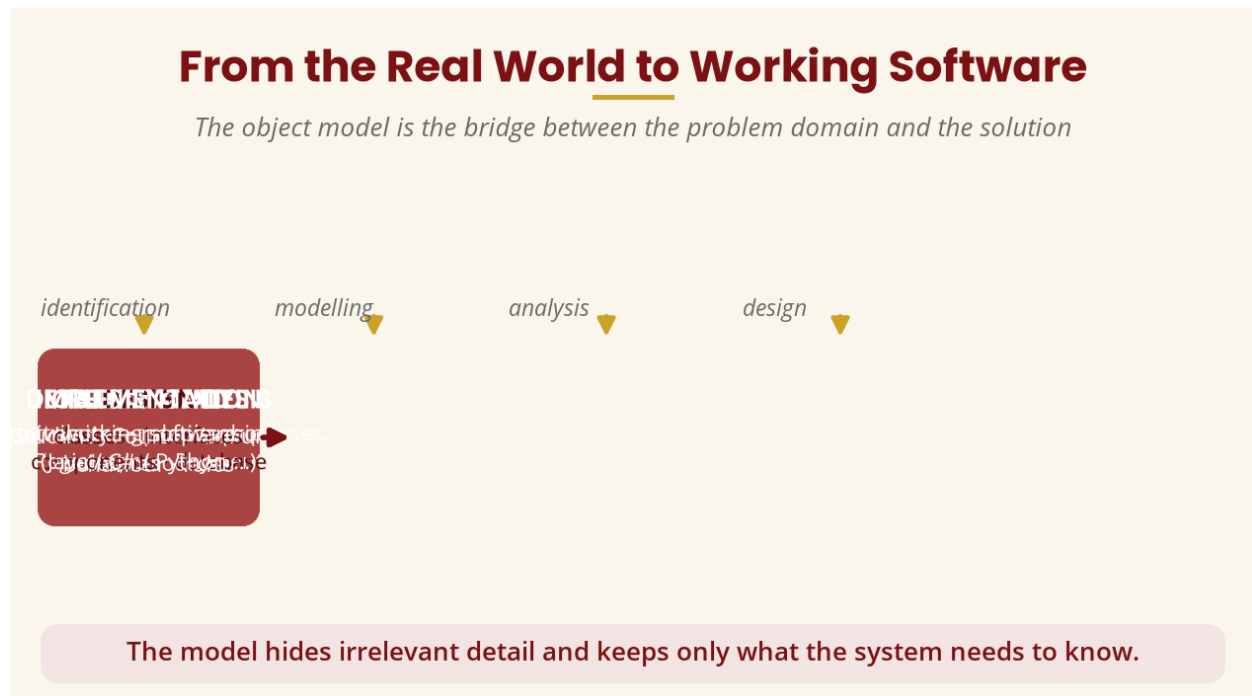


Figure 1 — The object model is the bridge between the real-world problem and the software solution

2. Learning Outcomes

By the end of this week, students should be able to:

- Explain the meaning of object modelling and define an object model.
- Explain why object models are useful in systems analysis.
- Explain the relationship between an object, a class and an object model.
- Identify potential objects in a given problem domain.
- Distinguish useful objects from irrelevant nouns.
- Identify attributes, behaviours and relationships associated with objects.
- Explain object identity, abstraction and encapsulation in relation to modelling.

- Explain how object models help manage system complexity.
- Explain the importance of consistency and standards in modelling.
- Construct a simple conceptual object model from a textual problem description.
- Evaluate whether a proposed object model adequately represents the business problem.

3. What Is Object Modelling?

Imagine explaining a university to somebody who has never seen it. You could try to describe everything — every student, lecturer, classroom, chair, computer, document, tree, road, employee, department and activity. That would be overwhelming.

Instead, you create a **simplified representation** showing only the things relevant to your purpose. For a student registration system you might focus on **Student, Course, Programme, Lecturer, Registration, Department and Payment**.

That simplified representation is a **model**. When we organise the model around objects and their relationships, we are performing **object modelling**.

DEFINITION

An **object model** is a representation of a system or problem domain in terms of objects, their characteristics, behaviour and relationships.

A First Object Model

An object model shows objects, their characteristics and their relationships



Student — registers for — Course (a simplified picture, not the whole university)

Figure 2 — A first object model: Student — registers for — Course, with key attributes

4. Why Do We Need Object Models?

“Build us an online student registration system” is too broad a statement. The developer must understand **who** registers, **what** is being registered, **who** approves, **which** courses are available, **what** information registration contains, whether prerequisites apply, whether payment affects registration, and who may change a registration.

Object modelling transforms a vague description into a structured, communicable representation.

Object modelling is a form of simplification. The real world is enormously complicated; a software model cannot — and should not — represent every detail. We select the details that matter. This is **abstraction**.

5. The Principles of Object Modelling

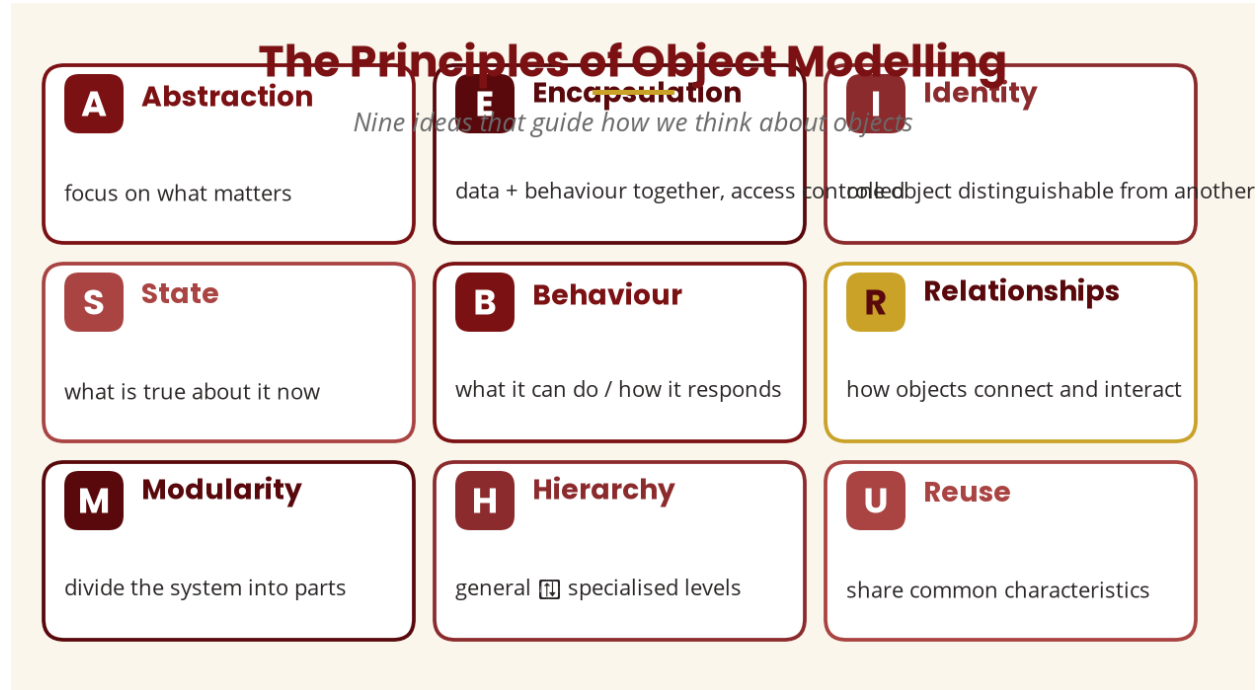


Figure 3 — Nine guiding principles of object modelling

5.1 Abstraction

Abstraction means focusing on the essential characteristics of something while ignoring details not relevant to the current purpose. A driver cares about steering, speed, fuel, brakes and gear — not the internal combustion process or every electrical connection.

A Student has hundreds of real-world characteristics (height, weight, hair colour, favourite food, shoe size, hobbies...). For a registration system the important attributes may be just **studentID**, **name**, **programme**, **year**, **status** — shoe size is irrelevant to course registration.

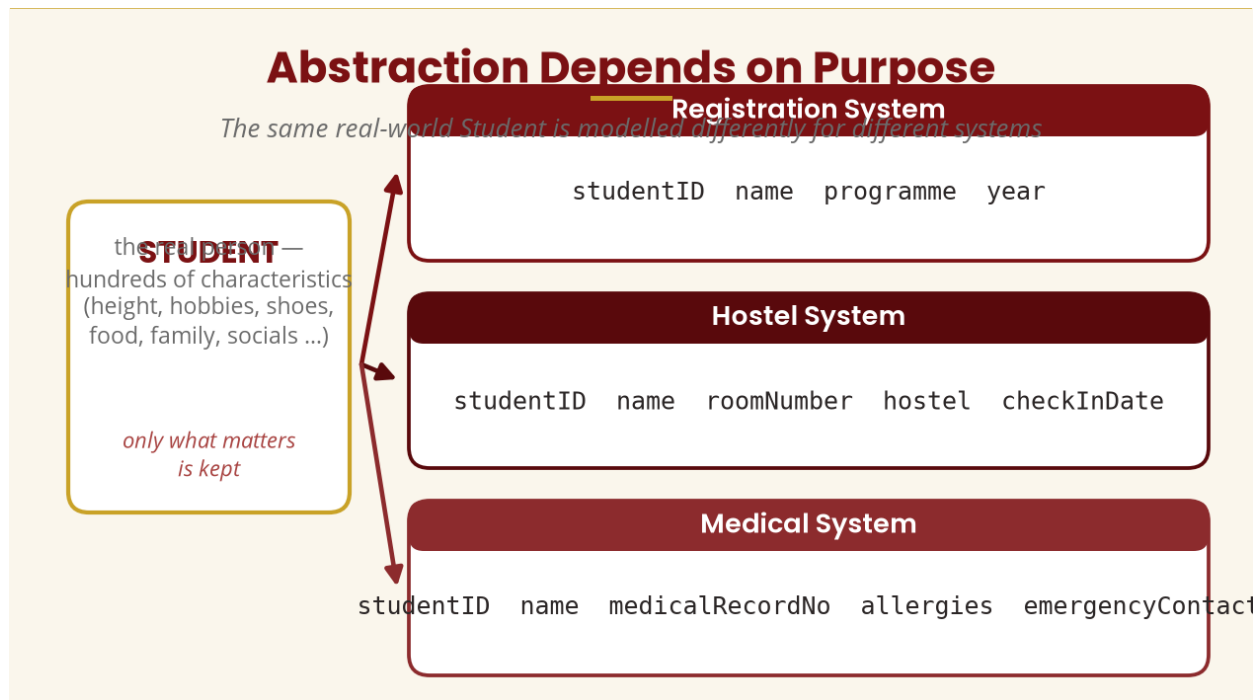


Figure 4 — Abstraction depends on purpose: the same Student is modelled differently for each system

5.2 Encapsulation

Encapsulation keeps an object's relevant data and operations together while controlling access to its internal state. A BankAccount owns its balance and provides deposit(), withdraw() and checkBalance() — instead of letting every part of the system write balance = -500000 without any checks.

5.3 Identity

Objects have **identity**. Two students may share the name “Francis” but remain different objects because their identifiers differ. Identity may be a student ID, account number, employee ID, product ID, UUID or database primary key.

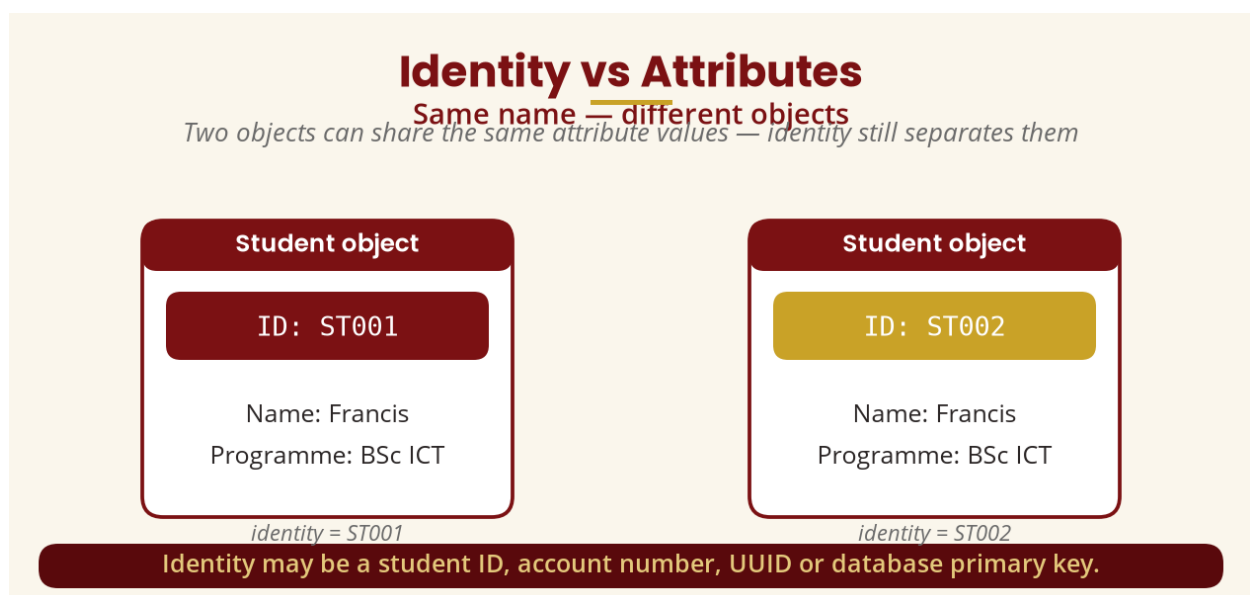


Figure 5 — Identity separates objects even when attribute values are identical

5.4 State

The **state** of an object is the collection of values describing its current condition (ID = ST001, Name = Francis, Year = 3, Status = Active). State can change over time — from Applicant, to Active Student, to Graduated. State represents what is true about an object at a particular moment.

5.5 Behaviour

An object's **behaviour** describes what it can do or how it responds to requests — a Student has registerCourse(), dropCourse(), viewResults(), payFees(); a BankAccount has deposit(), withdraw(), transfer(). So an object combines **identity + state + behaviour**.

5.6 Relationships

Objects rarely exist independently. Relationships connect them: **Student — registers for — Course, Lecturer — teaches — Course, Customer — owns — Account, Customer — places — Order**.

5.7 Modularity

Modularity divides a large system into manageable parts. A university information system can be split into Student, Finance, Exams and Library modules, each containing related objects (Finance holds Invoice, Payment, Account, Receipt).

5.8 Hierarchy

Hierarchy organises concepts into levels from general to specialised: **Person** □ **Student / Employee** □ **Lecturer / Administrator**. This connects directly to generalisation, specialisation and inheritance in later weeks.

5.9 Reuse

Good object models support reuse — a general Person (name, phone, email) can be shared by many parts of the system. But remember: **reuse does not mean “always use inheritance.”** Composition, delegation, interfaces and services can also support reuse.

6. Object Identification

Object identification means determining which entities or concepts in the problem domain should be represented as objects/classes in the system model. It sounds easy — it isn't.

From “A student registers for courses. The lecturer teaches the courses, and the finance department records the student's payments,” we might identify Student, Course, Lecturer, Finance Department, Payment and Registration — but we must not automatically accept every noun.

6.1 The Noun Technique

A traditional starting point is to examine the problem statement and underline important nouns. These become **candidate** concepts — but **candidate ≠ final object**.

KEY POINT

“The student uses a mobile phone to register for a course.” Should Phone be a class? Probably not — it may simply be an external device. Object identification requires **judgement, not just grammar.**

6.2 Classifying Candidate Objects



Figure 6 — Six common families of candidate concepts

Events can be objects. Payment is an event/transactional concept with paymentID, amount, date, method, status and behaviour such as process(), cancel(), refund() — a perfectly valid object even though it is not physical.

6.3 How to Decide

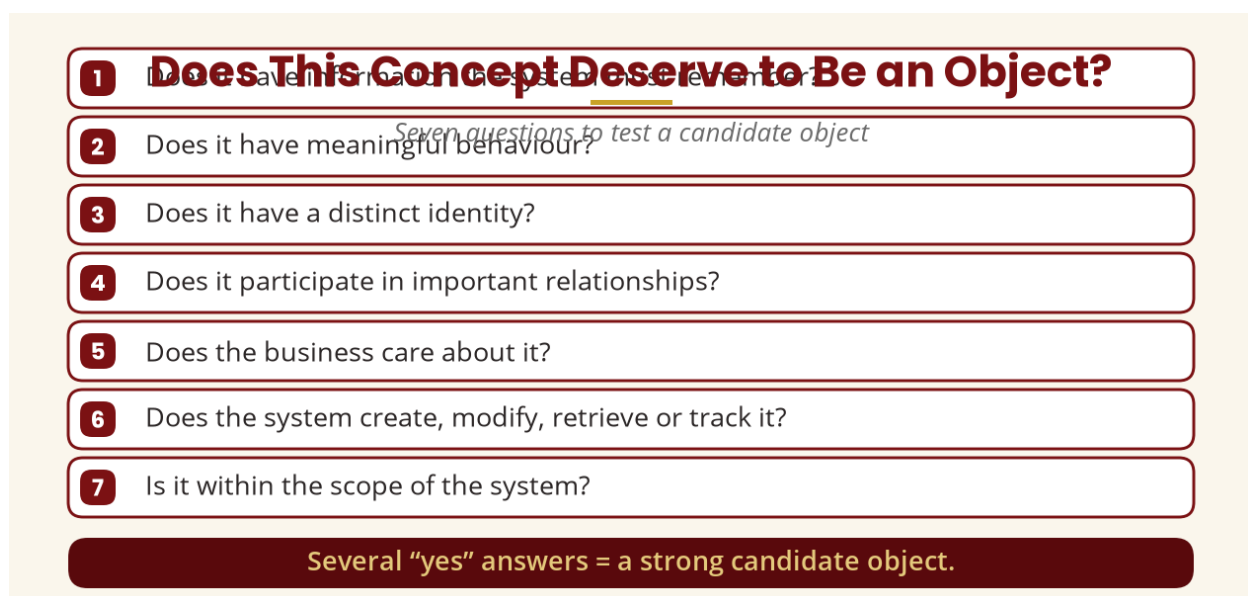


Figure 7 — Seven questions that test whether a concept deserves to be an object

6.4 Object vs Attribute

Deciding whether something is an object or merely an attribute is a common source of confusion. If the system only needs `phoneNumber = +265...`, then `phoneNumber` is an attribute. If it must manage multiple numbers, verification, ownership, type and status, a separate `Phone` concept may become useful. The decision depends on requirements and purpose.

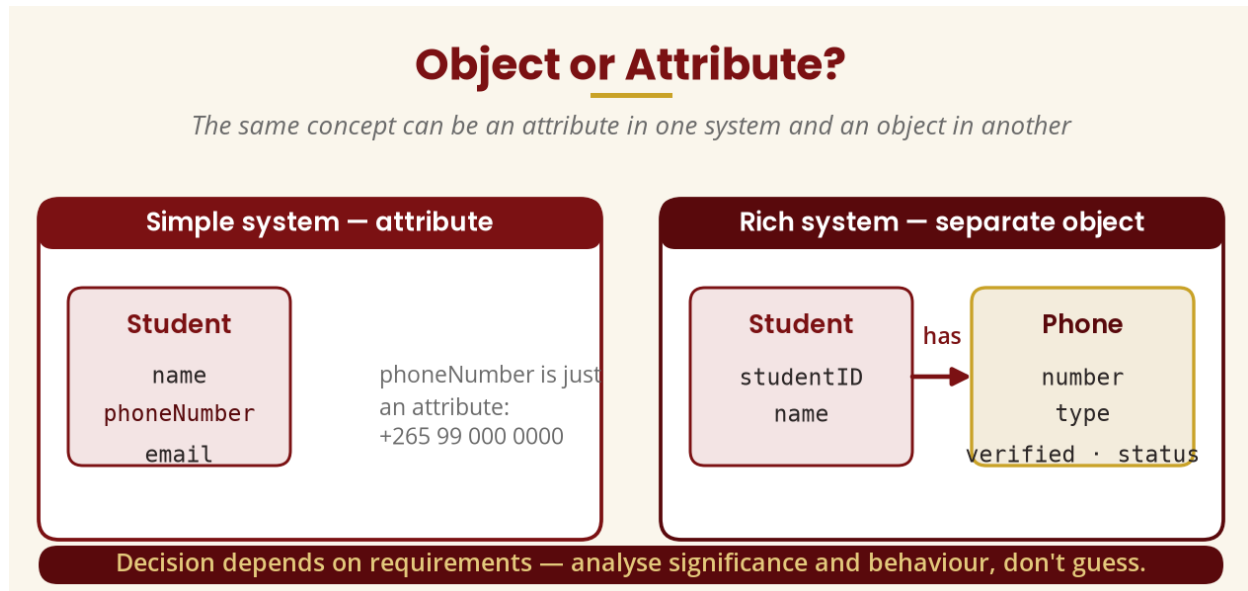


Figure 8 — The same concept can be an attribute in one system and an object in another

7. Attributes, Behaviour & Responsibility

Once candidates are identified, ask: **“What information must the system know about this object?”** (its attributes) and **“What does this object do?”** (its behaviour).

For `Student`: `studentID`, `name`, `dateOfBirth`, `programme`, `year`, `status` — with `register()`, `dropCourse()`, `viewResults()`. For `Payment`: `paymentID`, `amount`, `date`, `method`, `status` — with `process()`, `refund()`, `verify()`. Behaviour should be derived from the requirements, not invented.

OBJECT RESPONSIBILITY

Should a `Customer` calculate the interest on an `Account`? Probably not — `Account` is a better owner of that responsibility because it holds `balance`, `account type` and `interest rules`. **Give an object responsibilities that logically belong to it.**

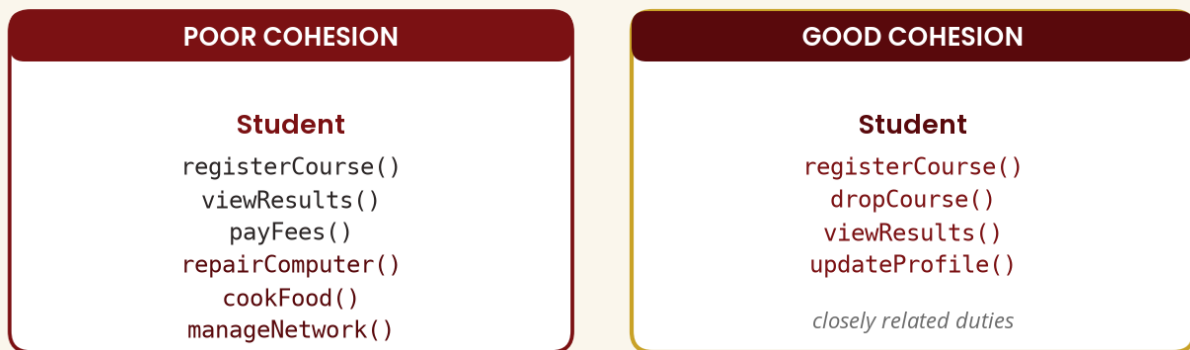
7.1 Cohesion and Coupling

Cohesion measures how closely related an object's responsibilities are. A `Student` with `registerCourse()`, `viewResults()`, `payFees()`, **`repairComputer()`**, **`cookFood()`** and **`manageUniversityNetwork()`** has poor cohesion.

Coupling is the degree of dependency between components. Well-designed systems aim for **high cohesion + manageable/low coupling**.

Cohesion — Keep Responsibilities Related

Poor cohesion mixes unrelated duties; good cohesion stays focused



Aim for high cohesion + manageable coupling — vital in object-oriented design.

Figure 9 — Poor cohesion mixes unrelated duties; good cohesion stays focused

8. Managing Complexity

A system with 10 classes is easy to understand; a system with 500 classes and many relationships is not. Object-oriented modelling uses several techniques to control this complexity.

Managing Complexity

Six techniques that keep large systems understandable

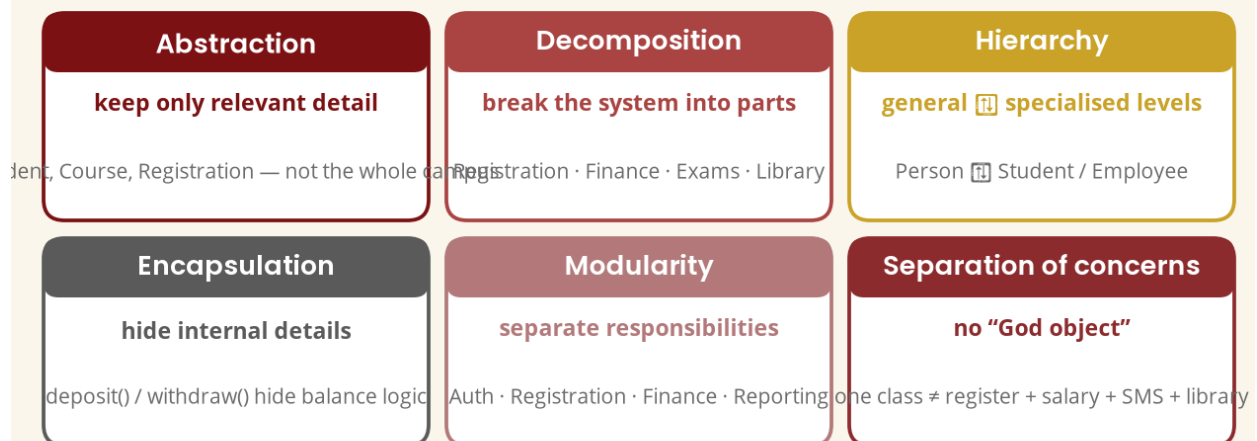


Figure 10 — Six techniques for managing complexity

Why Complexity Must Be Managed

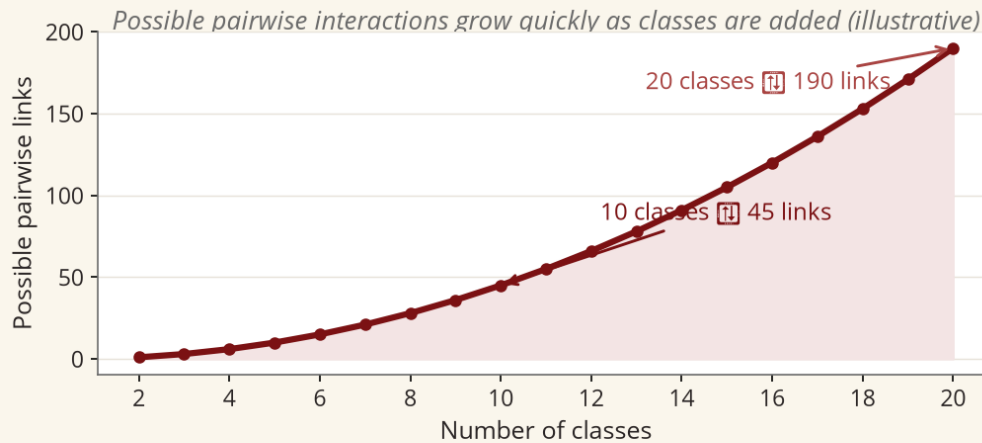


Figure 11 — Illustrative growth of possible interactions as classes are added

Separation of concerns deserves emphasis: avoid the “God object” — one component trying to do everything (registerStudent(), calculateSalary(), printInvoice(), repairComputer(), sendSMS(), calculateExamMarks(), manageLibrary()). Responsibilities should be distributed appropriately.

Poor Model vs Good Model

Everything in one box is not a model — separation is

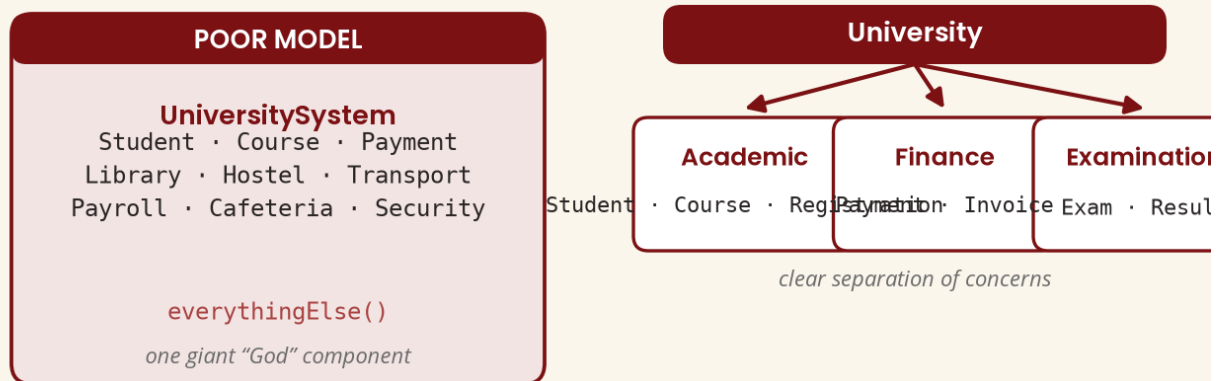


Figure 12 — A poor model throws everything into one box; a good model separates concerns

9. Modelling Viewpoints

A single model is often not enough to understand a complex system. We may want a **structural** view (what exists), a **behavioural** view (what happens), an **interaction** view (how objects communicate) and a **deployment** view (where components run).

Modelling Viewpoints

One model is not enough — we view a system from several angles

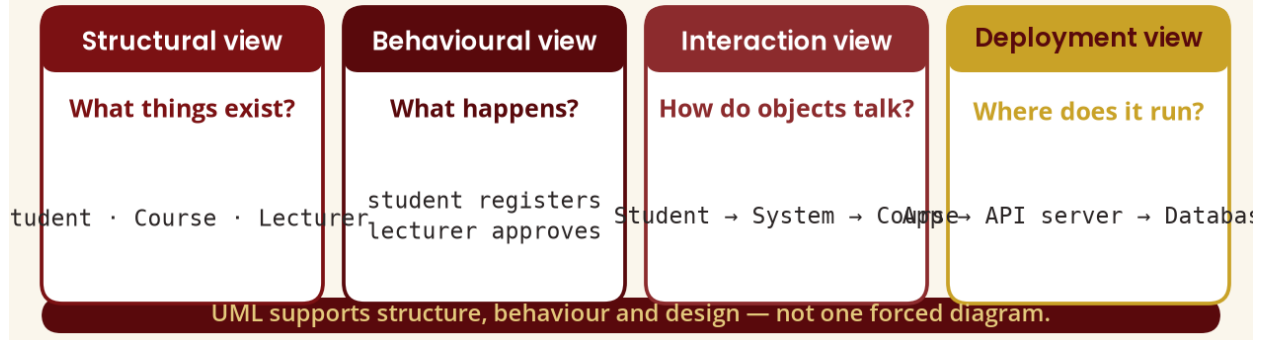


Figure 13 — UML supports several perspectives rather than forcing one diagram

10. Standards for Objects & Modelling

Software modelling should not be based on everyone's personal drawing style. Standard notation improves communication. For OOAD the major standard is the **Unified Modeling Language (UML)**, maintained by the Object Management Group; the currently published formal specification is **UML 2.5.1** (adopted December 2017).

10.1 Why Standards Matter

If three analysts draw a relationship as Student - - - - Course, Student <- -> Course and Student ===== Course, nobody knows what the symbols mean. A standardised language provides agreed meanings.

IMPORTANT

Standards do not replace thinking. A UML diagram can be perfectly drawn and still represent bad analysis — “Student owns Toilet” is syntactically correct but meaningless for a registration system. Correct notation does not guarantee correct analysis: understand the business, the requirements, select appropriate concepts, create meaningful relationships, then use correct notation.

11. Worked Examples

11.1 University Registration

Building our first conceptual object model from requirements proceeds in steps: **(1)** identify candidate objects (Student, Course, Lecturer, Registration, Programme, Payment); **(2)** identify attributes (Student: studentID, name, programme, year); **(3)** identify relationships (Student—Registration, Course—Registration, Lecturer—Course, Programme—Student); **(4)** identify behaviour (Student.registerCourse(); Registration.submit()/approve()/cancel(); Payment.process()/refund()).

11.2 Mobile Money System

For a mobile money system the candidate objects include **Customer**, **Wallet**, **Transaction**, **Agent**, **Merchant**, **Bill** — each with identity, state and behaviour.

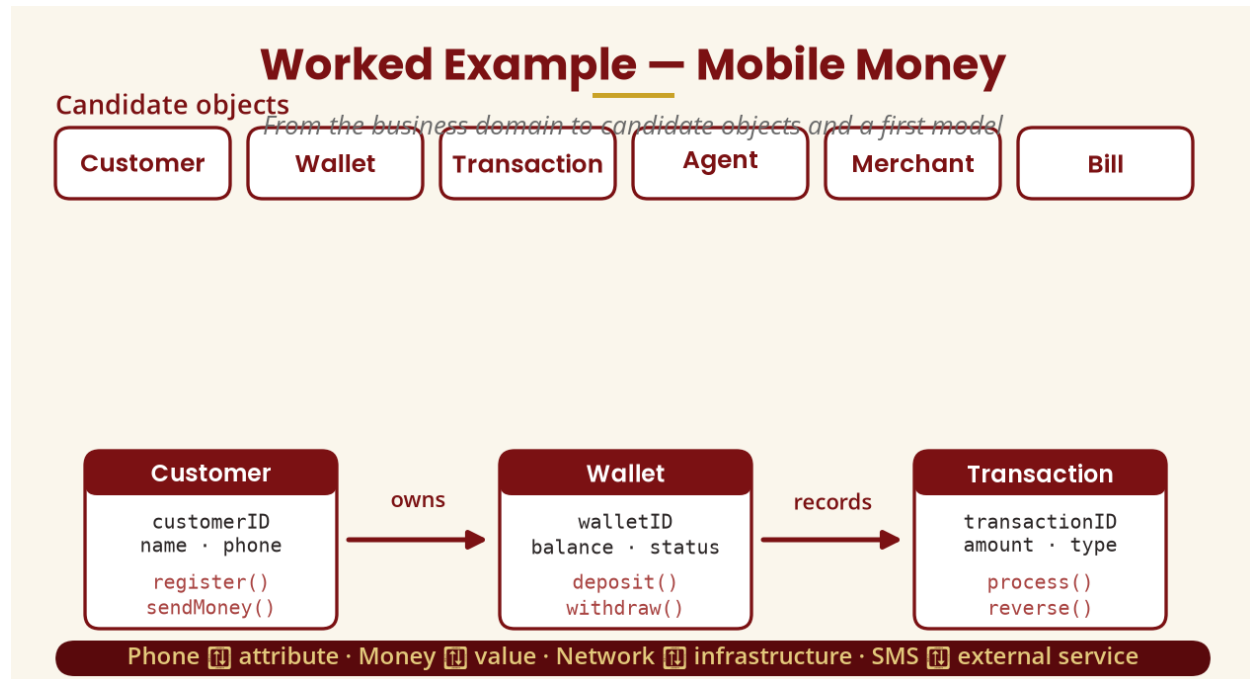


Figure 14 — Mobile money: candidates, attributes, behaviour and a first model

CANDIDATE vs FINAL

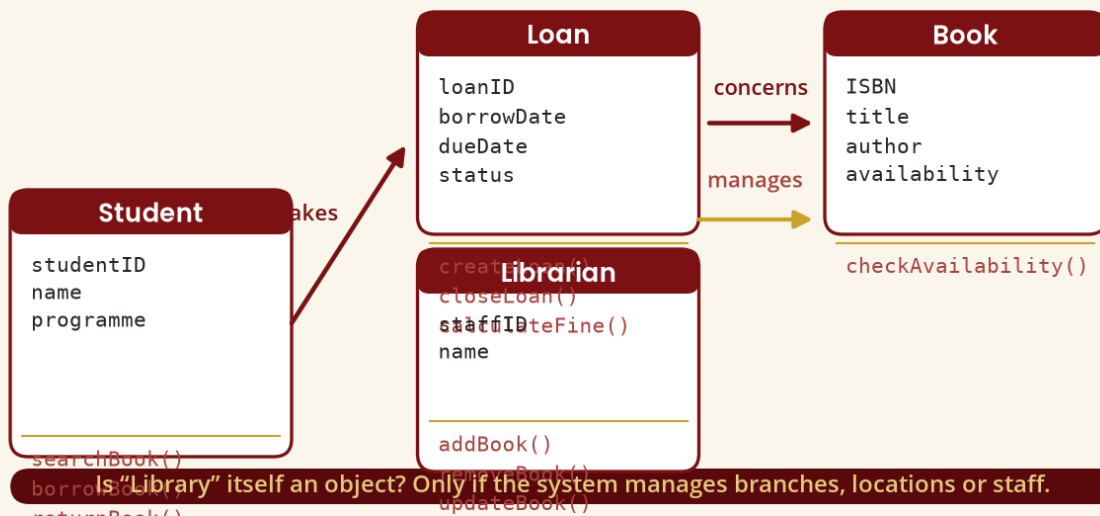
Initial analysis may list Customer, Phone, Money, Wallet, Transaction, Agent, Network and SMS — but we do not automatically create eight classes. Phone may be an attribute; Money is usually a value; Network is infrastructure; SMS may be an external service. **Object identification is analytical reasoning, not noun extraction.**

11.3 Mini Case Study — Online Library

A university wants an online library system: students search for, borrow and return books; librarians manage books and monitor overdue loans.

Mini Case Study — Online Library

Candidate objects, attributes, behaviour and relationships



Is "Library" itself an object? Only if the system manages branches, locations or staff.

Figure 15 — Library conceptual object model

Is "Library" an object? Possibly — but it depends on scope. If the system manages multiple libraries, branches, locations, hours and staff, then Library is meaningful; for a single library's borrowing process it may not be needed. **Scope and purpose determine the model.**

12. Object-Model Quality & Checklists

A good object model should be **relevant**, **understandable**, **consistent**, **sufficient**, **not unnecessarily detailed**, **traceable** to requirements, and **maintainable** as requirements change.

The Object-Identification Checklist



Figure 16 — The ten-step object-identification checklist for examination scenarios

12.1 Traceability from Requirements to Objects

A good analyst can answer: **“Where did this object come from?”** The requirement “Students must be able to register courses” leads to Student, Course and Registration; “the system must prevent registration when prerequisites are not satisfied” leads to a Course-prerequisite relationship. Requirements provide evidence for why model elements exist.

13. Practical & Tutorial Activities

13.1 Practical — Hospital Appointment System

Scenario: a hospital wants an electronic appointment system. Patients request appointments with doctors; doctors have specialties; receptionists manage appointments; the system records dates and statuses.

Task 1 — identify candidate objects (Patient, Doctor, Appointment, Receptionist, Specialty) and justify each.

Task 2 — identify attributes (Patient: patientID, name, phone; Doctor: doctorID, name, specialty; Appointment: appointmentID, date, time, status).

Task 3 — identify behaviours (Patient.requestAppointment()/cancelAppointment(); Doctor.viewAppointments(); Receptionist.scheduleAppointment()/rescheduleAppointment()).

Task 4 — identify relationships (Patient—requests—Appointment, Doctor—attends—Appointment, Doctor—has—Specialty, Receptionist—manages—Appointment).

13.2 Tutorial — “Is It an Object?”

For each candidate — Student, Name, Course, Mobile Phone, Payment, Money, University, Registration, Keyboard, Invoice, Lecturer, Internet — classify it as **likely object/class**, **likely attribute**, **external entity/infrastructure**, **depends on scope**, or **probably irrelevant**, and justify every decision.

TUTORIAL CHALLENGE

“A customer uses a mobile phone to purchase a product from an online shop, adds it to a shopping cart, pays using mobile money, and the system generates an invoice.” Identify objects, attributes, behaviours, relationships and external systems — and decide whether Mobile Phone, Mobile Money, Invoice and Shopping Cart should be objects. **The reasoning is the important part.**

14. Independent Learning (10 Hours)

- **Object identification** — choose a system (supermarket, bank, hospital, mobile money, university, hotel, library); write a one-page description and identify 15 candidate objects, 10 attributes, 10 behaviours and 10 relationships.
- **Object filtering** — from your 15 candidates, classify each as Keep/Remove with a reason (Student = keep, core entity; Keyboard = remove, implementation detail).

- **Complexity analysis** — for a large system, explain how abstraction, decomposition, modularity and hierarchy could reduce its complexity.

15. Week 2 Summary

Term	Meaning
Object modelling	Representing a system/problem domain using objects and their characteristics, behaviour and relationships.
Object	An identifiable entity with state and behaviour.
Class	A specification describing a type of object.
Abstraction	Focus on relevant information while hiding unnecessary detail.
Encapsulation	Keep data and related operations together while controlling access.
Identity	Allows one object to be distinguished from another.
State	The object's current condition.
Behaviour	What the object can do or how it responds.
Relationship	How objects are connected or interact.
Modularity / Hierarchy	Divide the system into parts / organise concepts general → specialised.
Object identification	Deciding which domain concepts belong in the model.
UML	Standardised modelling language; current OMG specification is UML 2.5.1.

THE MOST IMPORTANT LESSON

Object modelling is **not** turning every noun into a class. It is the disciplined process of identifying the important concepts in a problem domain, determining their relevant information and behaviour, and representing how they relate — moving from problem domain → candidate concepts → relevant objects → attributes → behaviour → relationships → an object model. Next, we need a standard language to communicate these models — which leads directly into **Week 3: UML**.

16. Examination-Style Questions

Question 1. Define object modelling and explain its importance in object-oriented analysis.

Question 2. Explain five principles of object modelling, illustrating each using a university management system.

Question 3. A university wants an online course registration system. Identify: (a) ten candidate objects, (b) five attributes for appropriate objects, (c) five behaviours, (d) five relationships, (e)

three examples of information to be excluded through abstraction.

Question 4. “Every noun in a requirements document should become an object.” Do you agree? Explain with examples.

Question 5. Explain how abstraction, decomposition, modularity, hierarchy, encapsulation and separation of concerns help manage complexity.

17. References and Further Reading

Standards and authoritative sources

- International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. ISO. (Current edition; reviewed and confirmed 2024.)
- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG.
- Object Management Group. *Unified Modeling Language (UML)*. OMG — UML as a standard language for visualising, specifying and documenting software, systems and business processes.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.